

## Lecture Homework Week 06 - Thursday Template

### Student Information

Name: Laiba Nasir

NetID: Inasi001

### Student Responses

Answer the following questions

#### Question 1

When running the Keyboard Demo in QEMU after introducing the bug, briefly describe how it behaves differently than before. Try a few keystrokes in the QEMU window and describe what you see:

**When running the keyboard demo in qemu after introducing the bug, it behaved differently because the screen kept glitching every time i tried to type something. The display was completely corrupted because it was messed up with visual artifacts. And when i tried to typing it, no keyboard input showed on the screen. Before this bug, it printed the full line after enter.**

```
18 | B .
19 |
20 | // broken bhvr - with bug
21 | irq_handler:
22 | sub lr, lr, #4
23 | stmfd sp!, {r0-r12, lr} // stack ALL registers
24 | bl IRQ_handler // call IRQ_handler() in C
25 | ldmfd sp!, {r0-r3, r12, pc}^ // return
26 |
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 2
root@codespaces-0a202f:/workspaces/Lecture-Homework-Week-06----Thursday# qemu-system-arm -M versatilepb -n
```



Question 2 What do you think is happening? The program continues to execute, but what happens after it returns from `irq_handler`? What C subroutine gets called over and over?

After it returns from `irq_handler`, the broken `Imfd` has executed. Then, the PC jumps to `0x0`, and this is the ARM reset vector which causes the processor to restart execution from the beginning. We call `main()` all over again because of this. So essentially, every time a keyboard interrupt fires, the handler corrupts the return address, and the program jumps to the reset vector and ends up calling `main()` again. This reinitializes our display, timer, and everything else, causing the visual artifacts in an endless loop. So, the C subroutine which gets called over and over again is `main()`.

Question 3 Look at the terminal where the buggy version is running, what messages are being printed out about the timers? Do the timers work in this buggy version?

- The messages are being printed out about the timers print:
  - "Timer with delta zero, disabling"
- Since the `main()` keeps getting called, the timer getting reinitialized, before it gets a chance to actually do the counting.
- The execution of this timer is like this:  
The timer starts, then immediately gets reset, and repeats like this, and never really functions correctly.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS ⓘ
O root@codespaces-0a202f1:/workspaces/Lecture-Homework-Week-06---Thursday# gdb-multiarch build/keyboard.elf
GNU gdb (Ubuntu 15.1-1ubuntu1-24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/keyboard.elf...
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
0x00000000 in ?? ()
(gdb) break irq_handler
Breakpoint 1 at 0x1001c: file /workspaces/Lecture-Homework-Week-06---Thursday/src/start.s, line 22.
(gdb) continue
Continuing.

Breakpoint 1, irq_handler () at /workspaces/Lecture-Homework-Week-06---Thursday/src/start.s:22
22      sub lr, lr, #4
(gdb) next
23      stmfd sp!, {r0-r12, lr} // stack ALL registers
(gdb) next
24      bl IRQ_handler // call IRQ_handler() in C
(gdb) next
25      ldmfd sp!, {r0-r3, r12, pc}^ // return
(gdb) step
0x00000000 in ?? ()
(gdb) info register pc
pc          0x0
(gdb) x/i 0x00000000
=> 0x0: ldr pc, [pc, #24] @ 0x20
(gdb) []
```

